

INTERNSHIP FINAL REPORT

# QuickTalk Organizational Memory & Agent Knowledge Platform

*Flask tool calling · Mem0 · Pinecone · Groq · Analytics*

**Project result** A working multi-tenant customer-support memory service that recalls prior conversations, creates concise agent handoffs, learns reusable human-agent knowledge safely, and exposes organization analytics.

| Prepared for                                | Prepared by              | Report date    | Repository status              |
|---------------------------------------------|--------------------------|----------------|--------------------------------|
| Internship supervisor /<br>engineering lead | Internship project owner | 27 August 2026 | Main branch · 48 tests passing |

CONFIDENTIALITY NOTE

API keys are intentionally excluded. Any key previously shared in chat must be rotated before production use.

## 1. Executive Summary

QuickTalk is a Flask-based support intelligence layer that gives a bot and human-agent team controlled access to organizational and customer conversation memory. The project connects live chat, cross-session recall, agent handoff, reusable human-agent knowledge, tone guidance, and analytics while keeping every lookup scoped to an organization and customer identity.

| Capability            | Implemented outcome                                    | Primary technology                                 |
|-----------------------|--------------------------------------------------------|----------------------------------------------------|
| Conversational memory | Cross-session semantic recall for a single customer    | Mem0 + Pinecone + Ollama embeddings                |
| Human handoff         | Three concise bullets and expandable session summaries | Flask + precomputed profiles                       |
| Knowledge fallback    | Bot KB first, approved agent knowledge second          | Versioned SQL state + Pinecone candidates          |
| Analytics             | Issues, sentiment, resolution and missing-topic trends | Structured event store + deterministic aggregation |
| Organization tone     | Style-only rewrite without changing facts              | Groq with safe fallback                            |

**Current status** Locally demonstrable, committed to GitHub, real Mem0/Pinecone save-search verified, and 48 automated tests passing. It is a production-oriented prototype, not yet a production deployment.

## 2. Problem and Objectives

Customer-support conversations often restart from zero. Agents reread long histories, bots forget prior context, useful resolutions remain trapped in transcripts, and managers cannot see which topics the bot fails to answer. The project addresses these gaps through four objectives:

- Remember useful customer context across sessions without mixing tenants or customers.
- Give a human agent a short, actionable handoff instead of a transcript wall.
- Turn only complete and reusable human-agent answers into governed organization knowledge.
- Convert conversation events into operational analytics and missing-knowledge priorities.

### 3. System Architecture

The platform separates language generation, semantic retrieval, and authoritative business state. This is the central architectural decision.

| Layer          | Responsibility                                                | Does not do                                        |
|----------------|---------------------------------------------------------------|----------------------------------------------------|
| Browser UIs    | Chat, handoff, knowledge review, analytics and tool testing   | Store provider secrets                             |
| Flask API      | Validate, authenticate, enforce scope and dispatch tools      | Act as a vector database                           |
| Mem0           | Extract and retrieve conversational memories                  | Own article lifecycle or dashboard totals          |
| Pinecone       | Store organization-namespaced vectors for semantic search     | Transactions, joins or authoritative version state |
| Groq           | Summarization, curation and tone language tasks               | Persistent storage                                 |
| SQLite demo    | Messages, profiles, summaries, analytics, versions and audits | Production-scale distributed operation             |
| Redis optional | Shared low-latency profile cache in production                | System of record                                   |

#### 3.1 Request flow

1. User action reaches a Flask page or JSON endpoint.
2. Flask validates service key, required fields, role and organization scope.
3. The tool registry chooses the correct memory, knowledge, handoff or analytics operation.
4. Mem0/Pinecone or structured SQL retrieves evidence; Groq is called only where language generation is needed.
5. The final response returns with grounding metadata and is stored as conversation history.

#### 3.2 Tenant isolation

- Pinecone namespace is derived from organization scope.
- Customer Mem0 identity is an opaque hash of organization plus normalized mobile number.
- Relational queries always include organization scope.
- Flask rejects a mismatch between the claimed organization header and tool arguments.
- Production must derive organization and role from signed authentication claims.

## 4. Tool Catalog and Activation

| Tool                           | Activates when                               | Outcome                                                      | Core dependency                  |
|--------------------------------|----------------------------------------------|--------------------------------------------------------------|----------------------------------|
| save_customer_memory           | After every customer or assistant message    | Writes conversation memory and analytics event               | Mem0/Pinecone + SQL              |
| search_customer_memory         | Explicit personal-history question           | Cross-session semantic recall; general questions are skipped | Mem0/Pinecone + summary fallback |
| get_customer_memory_context    | New session initialization                   | Returns counts, current issue and recent summaries           | Precomputed profile              |
| get_contextual_welcome         | Chat opens / identity changes                | Greets returning customer using latest useful issue          | Profile + Groq tone              |
| get_handoff_context            | Escalation to human                          | Returns three bullets, counts and expandable sessions        | Profile + session summaries      |
| get_organization_tone_profile  | Every customer-facing response / manual test | Returns style guidance only; no facts                        | Agent-message analysis           |
| get_knowledge_curation_policy  | Agent chat closes / admin inspection         | Returns tenant-specific save and reject rules                | Versioned policy state           |
| search_approved_knowledge      | Bot KB miss / manual test                    | Searches active current-version agent knowledge              | Pinecone + SQL status gate       |
| resolve_support_answer         | General support question                     | Bot KB → approved agent knowledge → apology                  | Knowledge orchestrator           |
| get_missing_knowledge_topics   | Dashboard/report request                     | Aggregates unanswered topics without an LLM call             | SQL aggregation                  |
| import_agent_history_from_json | Explicit admin backfill                      | Starts controlled historical ingestion                       | Background importer              |

## 5. Detailed Tool Behavior

### save\_customer\_memory

Inputs: organization, session, mobile, role, text, optional timestamp. The service normalizes identity, writes through Mem0/Pinecone, records a structured event, and refreshes the customer profile. Failure is surfaced without exposing provider keys.

### search\_customer\_memory

Runs only when the query explicitly refers to personal history. It searches across sessions, not only the current session. Results are combined with tenant/customer-scoped session summaries so incomplete vector backfills cannot break recall. The server decides answer eligibility.

### get\_customer\_memory\_context

Loads a compact starting context for orchestration: memory count, previous-session count, current issue and recent summaries. It avoids sending the entire transcript to an LLM.

### **get\_contextual\_welcome**

Uses the latest meaningful issue to ask whether it was resolved. When no history exists, it returns a neutral greeting. Tone rewriting has a short timeout and cannot alter facts.

### **get\_handoff\_context**

Uses a precomputed profile for speed. The top of the inbox shows concise session summaries and counts; the full transcript loads only when the agent expands a session.

### **get\_organization\_tone\_profile**

Learns reply-length, politeness and language-style signals. It explicitly marks `facts_learned=false`, keeping voice separate from knowledge.

### **get\_knowledge\_curation\_policy**

Defines what can become reusable organizational knowledge. Controlled prices and regulations require scope, conditions and effective/version context.

### **search\_approved\_knowledge**

Pinecone supplies semantic candidates, but SQL verifies organization, active status and exact current article version. Disabled, deleted or superseded vectors are rejected.

### **resolve\_support\_answer**

Search order is bot knowledge first, approved agent knowledge second, safe apology third. A total miss creates a knowledge-gap event for analytics.

### **get\_missing\_knowledge\_topics**

Groups questions that neither knowledge source could answer. Counts are deterministic database results; an LLM is not required for authoritative totals.

### **import\_agent\_history\_from\_json**

Administrative backfill for historical transcripts. It is not called during normal chat. Production should move this work to a queue with job IDs, idempotency and dead-letter handling.

## 6. Main End-to-End Workflows

### 6.1 Live chat and RAG decision

1. Customer asks a question.
2. If it explicitly references personal history, search that customer's Mem0/Pinecone memories across sessions.
3. Otherwise search the organization's bot knowledge base.
4. If bot knowledge misses, search active approved human-agent knowledge.
5. If both miss, apologize safely and record the missing topic.
6. Apply organization tone to the grounded answer and store both messages.

**Important relevance rule** A question such as "Is weekend installation available?" is a general policy question and must not be answered from unrelated personal memories. A question such as "What internet problem did I report previously?" is routed to personal memory.

### 6.2 Human-agent handoff

1. Agent clicks Escalate to human.
2. Flask loads the precomputed customer profile and latest session summaries.
3. The card displays concise history, session count and memory count.
4. Older sessions and full transcripts remain expandable, reducing initial reading time.

### 6.3 Human-agent knowledge lifecycle

1. Human-agent transcript is stored under the organization.
2. When the chat closes, Groq proposes a standalone reusable question and answer.
3. A deterministic validator rejects filler, referrals, incomplete answers and customer-specific data.
4. Accepted content becomes an active versioned article and is indexed semantically.
5. Editing creates a newer version; disabling or deleting prevents retrieval immediately through the SQL status gate.

## 7. Memory, Knowledge and Tone Boundaries

| Information type               | Correct home                         | Example                                          |
|--------------------------------|--------------------------------------|--------------------------------------------------|
| Customer conversational memory | Mem0/Pinecone                        | Previously discussed doctor, issue or preference |
| Reusable organization fact     | Versioned knowledge + Pinecone index | Complete installation policy or current price    |
| Authoritative lifecycle state  | SQL                                  | Article active version, deletion and audit trail |
| Analytics event                | SQL / warehouse                      | Category, sentiment, resolution or gap           |
| Writing style                  | Tone profile                         | Concise, polite, Roman Urdu usage                |

## 8. Data Model and Indexing

| Table group              | Tables                                                                 | Purpose                                            |
|--------------------------|------------------------------------------------------------------------|----------------------------------------------------|
| Organizations and memory | organizations, memory_events                                           | Tenant metadata and normalized conversation events |
| Customer acceleration    | customer_profiles, session_summaries, durable_facts                    | Fast handoff, welcome and recent-context access    |
| Gap analytics            | knowledge_gap_events                                                   | Questions the bot could not answer                 |
| Agent source records     | agent_sessions, agent_messages                                         | Original organization-owned human transcripts      |
| Governed knowledge       | knowledge_articles, knowledge_article_versions, bot_knowledge_articles | Approved content, active versions and bot KB       |
| Governance               | knowledge_audit_log, knowledge_curation_policies                       | Change history and tenant-specific rules           |

Indexes cover organization plus time, session, mobile, category, status, gap state, durable-fact lookup and article/version status. These indexes support the dashboard and handoff without scanning full histories.

### 8.1 Why Pinecone is not the only database

- Vector search is optimized for semantic similarity, not transactions and joins.
- Dashboards need exact grouped counts and time filters.
- Knowledge lifecycle needs authoritative active/deleted/version state.
- Auditability needs immutable source records and change logs.
- Production therefore uses Pinecone for retrieval and PostgreSQL/warehouse storage for authoritative business data.

## 9. Analytics

Every event is tenant- and time-scoped. The dashboard can show contact volume, categories, sentiment, resolution state, unresolved sessions and missing knowledge. Groq may help classify or summarize language; database aggregation remains authoritative.

| Metric                | How it is produced                    | Management use                       |
|-----------------------|---------------------------------------|--------------------------------------|
| Issue/category volume | Grouped structured events             | Staffing and service priorities      |
| Resolution rate       | Session outcome/status                | Quality and escalation monitoring    |
| Sentiment             | Per-session/message classification    | Identify frustration and risk        |
| Missing topics        | Only total bot+agent-knowledge misses | Plan new KB training                 |
| Repeat contacts       | Customer/session counts               | Detect unresolved recurring problems |

## 10. Security and Governance

- Provider and service keys remain server-side in an ignored `.env` file.
- Organization namespace and normalized customer identity are applied to every memory operation.
- Role checks protect human-agent and administrator actions.
- Knowledge is filtered through curation plus deterministic validation.
- Version/status verification prevents stale Pinecone vectors from being used.
- Keys exposed in messages must be rotated before production.
- Production needs signed auth claims, TLS, rate limiting, retention/deletion policy and secrets management.

## 11. Reliability and Performance

- Profiles and summaries are recomputed on new events rather than on every handoff.
- Local TTL cache accelerates the demo; Redis is the production shared-cache option.
- Mem0/Pinecone cold initialization has a bounded timeout; structured session summaries preserve continuity during provider delays.
- Tone has a short timeout and returns the original grounded answer if unavailable.
- SQL indexes support tenant/time and profile/session lookups.
- Production should add queue/outbox synchronization, retry limits, reconciliation and dead-letter handling.

## 12. Testing Evidence

| Evidence                 | Result                                                            |
|--------------------------|-------------------------------------------------------------------|
| Automated suite          | 48 tests passing                                                  |
| Real Pinecone index      | Ready, cosine metric, 768 dimensions                              |
| Real Mem0 round trip     | Memory saved and retrieved from a different session               |
| Cross-session regression | Current session no longer filters out earlier sessions            |
| Isolation                | Wrong organization/mobile cannot retrieve customer history        |
| Negative validation      | Bad keys, fields, roles, timestamps and scope mismatches covered  |
| Knowledge governance     | Rejection, version activation and disable/delete behavior covered |

**Verified example** The test query “What was my previous verification code word?” returned the exact saved Mem0/Pinecone vector from an older session, alongside the structured-summary resilience source.



## 13. Boss Demo Script

| Step | Page        | Action                                                    | What to explain                           |
|------|-------------|-----------------------------------------------------------|-------------------------------------------|
| 1    | /tools      | Show 11 callable schemas                                  | The service is tool-callable, not UI-only |
| 2    | /custom     | Ask a normal policy question                              | Bot KB is searched before agent knowledge |
| 3    | /custom     | Ask: What internet problem did I report previously?       | Mem0/Pinecone cross-session recall        |
| 4    | /custom     | Escalate to human                                         | Concise handoff with expandable sessions  |
| 5    | /knowledge  | Open an agent transcript and article versions             | Strict reusable-knowledge governance      |
| 6    | /dashboard  | Show categories, resolution, sentiment and missing topics | Memory becomes operational insight        |
| 7    | /api/health | Show active backend                                       | mem0-groq-pinecone and cache status       |

### 13.1 Suggested 60-second explanation

“I built a Flask tool-calling service that gives a support bot safe organizational memory. Mem0 orchestrates cross-session memory, Pinecone stores tenant-isolated vectors, Groq handles language tasks, and SQL keeps authoritative profiles, analytics and knowledge versions. The bot searches personal history only for personal-history questions; general questions use bot knowledge first and approved human-agent knowledge second. Human handoff shows concise session summaries, and unanswered topics become dashboard priorities. The implementation has 48 passing tests and a verified real Mem0/Pinecone round trip.”

## 14. Production Roadmap

| Priority | Change                                            | Reason                               |
|----------|---------------------------------------------------|--------------------------------------|
| P0       | Rotate exposed keys and use a secrets manager     | Credential safety                    |
| P0       | Signed authentication claims for tenant/user/role | Prevent client-controlled scope      |
| P0       | PostgreSQL plus migrations and backups            | Concurrency, durability and scale    |
| P1       | Queue/outbox for Pinecone and imports             | Reliable synchronization and retries |
| P1       | Redis shared cache                                | Multi-worker performance             |
| P1       | Metrics, tracing, alerts and provider SLAs        | Operational visibility               |
| P2       | Retention, deletion and consent workflows         | Privacy and compliance               |
| P2       | Evaluation dataset and relevance monitoring       | Measure RAG quality over time        |

## 15. Limitations and Honest Status

- The current database is SQLite and is intended for demonstration, not multi-instance production.
- The Flask development server must be replaced by a production WSGI deployment.
- External providers can have cold starts, rate limits and outages; fallbacks reduce impact but do not remove dependency risk.
- Historical backfill quality depends on source transcript quality and strict curation.
- Analytics classification should be evaluated against labeled organization data before business decisions depend on it.
- Browser-side demo authentication must be replaced by real sessions and claims.

## 16. Internship Outcomes

- Designed a multi-tenant memory and retrieval architecture.
- Integrated Flask tool calling with Mem0, Pinecone, Groq and Ollama.
- Implemented relevance routing between personal memory, bot KB and agent knowledge.
- Built governed, versioned knowledge from human-agent transcripts.
- Created optimized handoff profiles and organization analytics.
- Diagnosed real integration failures, added resilience and regression tests.
- Documented production gaps rather than presenting a prototype as fully production-ready.

## 17. Glossary

| Term               | Plain-language meaning                                                           |
|--------------------|----------------------------------------------------------------------------------|
| RAG                | Retrieve relevant evidence before generating an answer                           |
| Mem0               | Memory orchestration library for extracting and recalling conversational facts   |
| Embedding          | Numeric representation that allows search by meaning                             |
| Pinecone namespace | Organization-isolated vector partition                                           |
| Tenant             | One organization using the shared platform                                       |
| Handoff            | Context transferred from bot to human agent                                      |
| Knowledge gap      | A question neither approved knowledge source could answer                        |
| Outbox             | Reliable event pattern for synchronizing database changes with external services |

## Appendix A. Configuration Required

| Variable               | Purpose                        |
|------------------------|--------------------------------|
| SERVICE_API_KEY        | Protect tool and memory routes |
| MEMORY_BACKEND         | Select mem0-groq               |
| PINECONE_API_KEY       | Authenticate vector storage    |
| MEMO_PINECONE_INDEX    | Select index                   |
| GROQ_MEMORY_API_KEY    | Mem0 extraction                |
| GROQ_SUMMARY_API_KEY   | Session summaries and welcomes |
| GROQ_KNOWLEDGE_API_KEY | Knowledge curation and tone    |
| OLLAMA_BASE_URL        | Local embeddings               |

|                            |                          |
|----------------------------|--------------------------|
| MEMO_LOCAL_EMBEDDING_MODEL | nomic-embed-text:latest  |
| ANALYTICS_DB_PATH          | Structured demo database |

No real credential value is included in this report.

## Appendix B. Main Demo URLs

- Customer chat and handoff: <http://127.0.0.1:8765/custom>
- Human-agent knowledge: <http://127.0.0.1:8765/knowledge>
- Organization analytics: <http://127.0.0.1:8765/dashboard>
- Tool catalog: <http://127.0.0.1:8765/tools>
- Health: <http://127.0.0.1:8765/api/health>